7. Table Guide

Contents

New terms 1
Table arguments 2
- `columns` 3
- `fill` 3
- `stroke` 3
- `inset` 3
- `gutter`, `row-gutter`, `column-gutter` 4
- align 4

Table elements 4
- `table.header` and `table.footer` 4
- `table.cell` 4
- `table.hline` and `table.vline` 4

Programming 5
- `calc` helpers 5
- Arrays 5

Content wrappers 6
- `#rotate(...)[...]` 6
- `#figure(...)[...]` 7
- `#grid()` 7

Questions 7

New terms

Term	Definition	Examples
Sides dictionary	a dictionary with `x, y, top, bottom, right, left, rest` keys	`(x: 1cm)` | `(left: red, bottom: 2pt + blue)`
Joined value	a value produced by combining values of different type with `+` operator	`1pt+red` | `bottom + center`

Term	Definition	Examples
Binding	creating an association between a name and an entity	`#let a = 2` | `#let square(a) = a*a` | `#let (.., x) = ("22", "13", "14")`
Discarding	pattern matching where we use underscore or `..` to intentionally ignore certain values without binding them to variables	`#let (x, ..) = (1, 2, 3)` | `(_, y) => y-1`

CSV	comma-separated values; a format for table representation – one row per line with the cells separated by commas	`Name, Score`
		`Ann, 90`
		`Bob, 95`

Table arguments

Each table cell have (x, y) coordinates. We can use this coordinates in binary closure or `table.cell.where(x: <n>)` and `.where(y: <n>)` selectors.

0, 0	1, 0	2, 0	3, 0
0, 1	1, 1	2, 1	3, 1
0, 2	1, 2	2, 2	3, 2
0, 3	1, 3	2, 3	3, 3
0, 4	1, 4	2, 4	3, 4

Most table arguments can accept several forms of values:

Form of value	Description	Examples
Single value	applies to everything	`fill: red | align: right + horizon | stroke: none`

Array	applies iteratively behaves differently depending on the argument (by columns as default)	`fill: (red, blue, green) | align: (left, right) | gutter: (1pt, 3pt, 0pt)`
Sides dictionary	applies to cell sides can take binary closure as sub-argument	`inset: (x: 4pt, bottom: 3pt) | stroke: (top: none, rest: red)`
Binary closure	applies to conditionally specified cells, must return one of the above types	`(_, y) => if y == 0 {none} | (x, y) => if x > 3 or y < 4 {horizon} else {bottom} + if x < 6 {left} else {right}`

`#if false` without `else` evaluates to `none` which some arguments (`fill, stroke`) accept but others (`align, inset`) do not.

`columns`

- `auto` is the default size.

`fill`

- an array cycles by columns.
- takes `none`.

`stroke`

- an array cycles by columns.
- allows joined values `<color> + <length>`.
- allows `none` and sides dictionary.
- cell to the right or below overrides neighbor's stroke.

`inset`

- controls padding inside the sells.

- can be given as relative length (`em`, `%`) in any form (array, side dictionary, etc.)

`gutter`, `row-gutter`, `column-gutter`

- take length and push cells away from each other.
- array keep the last element till the end.
- `row-gutter` array iterates by rows.

align

- an array cycles by columns.

Table elements

`table.header` and `table.footer`

- can wrap several content blocks: `[...][...][...]...`
- repeat at the beginning/end of a page when table splits unless `(repeat: false)`.
- cannot be accessed thru their own `#show` selectors, use `table.cell.where(y: <num>)` instead.

`table.cell`

- produces and sets a single cell.
- has some table arguments, overriding them: `fill, stroke, align, inset`.
- `colspan: <num>` and `rowspan: <num>` arguments make the cell span across several columns/rows. The settings of the first cell in the span are applied.
- can be a selector for `#show` and adjusted thru `#set`.

`table.hline` and `table.vline`

- produce a *hand-placed line* independent from *grid-line* of `table.stroke`.
- have `stroke` argument controlling width and color.
- have `y: <num>` and `x: <num>` arguments respectively to place them not where they are written. `x: 0` is the leftmost line, `y:0` is the topmost.

- have `start: <num>` and `end: <num>` arguments, where `<num>` is a cell coordinate.
- can be adjusted thru `#set`.

Programming

Logical operators `or`, `not`, `and` behave as you expect.

You can refer to a variable in itself: `#let x = x + 1`.

`#(...)` evaluates an expression in markup: `#(1 + 2)` → 3.

`#{...}` allows to scope definitions and `#set` rules and even use definitions and multiple steps in closures:

```
#(1, 2, 3).map(x => {
  let double = x * 2
  double + 1
})
```
⟶ (3, 5, 7)

`calc` helpers

- `#float(<str>)` turns a string into number: `#float("228")` → 228.
- `#calc.odd(<num>)` and `#calc.even(<num>)` – `true` when the number is odd/even.
- `#calc.rem(<num1>, <num2>)` – returns the remainder of dividing the first number by the second: `#calc.rem(7, 2)` → 1.
- `#calc.round(<num1>, digits: <num2>)` – rounds `<num1>` to `<num2>` digits after dot: `#calc.round(3.145454, digits: 2)` → 3.15.
- `#calc.log(<num>)` – returns logarithm of `<num>` (base 10 by default), the base can be specified with `base:` argument:
  - `#calc.log(1000)` → 3;
  - `#calc.log(1000, base: 2)` → 9.965784284662087.

Arrays

Array indexes begin from 0.

- `<array>.at(<num>)` – returns an array element with index `num`:

`#(1, 2, 3).at(0)` → 1

This can be used to cycle array by rows:

```
fill: (_, y) => (red, green, yellow).at(calc.rem(y, 3))
```

- `.flatten()` – unnest nested arrays into 1-level array:

  `#(1, (2, 3, (4,))).flatten()` → `(1, 2, 3, 4)`

- `.slice(<num>)` – throws away all elements with index below `num`,
- `.slice(<num1>, <num2>)` – throws away all elements with index below `num1` and ≥ than `num2`:

  `#("a", "b", "c", "d", "e").slice(3)` → `("d", "e")`
  `#("a", "b", "c", "d", "e").slice(2, 3)` → `("c",)`
  The number of items you get is always **`num2 − num1`**.

- `#csv("file.csv")` turns the file into 2-level array where 1st level is rows and the 2nd – cells of a row in string form:

  ```
  (("This", "Is", "Row", "1"),
  ("This", "is", "second", "1"), ...)
  ```

- Here are some destructuring techniques:

  `#let (_, b, _, d) = (1, 2, 3, 4)` ⟹ `#b` → 2, `#d` → 4

  `#let (a, b, ..) = (1, 2, 3, 4)` ⟹ `#a` → 1, `#b` → 2

  `#let (.., a, b, c) = (1, 2, 3, 4, 69)` ⟹ `#a` → 3, `#b` → 4, `#c` → 69

Content wrappers

`#rotate(...)[...]`

- takes angle value `<num>deg` or `<num>rad` as positional argument. Positive `num` rotates clockwise.
- can take `reflow: true` makes rotation affect layout.

`#figure(...)[...]`

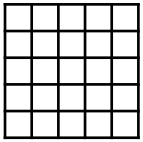
- takes `placement: top/bottom/none/auto` argument that moves it across page.
- puts its content inside a block, that makes the content unbreakable
  - a table inside a figure cannot span across pages
  - to change it, use `#show figure: set block(breakable: true)`

`#grid()`

- `table` is for data, `grid` is for layout.
- have the same arguments as `#table`
- have different defaults: no strokes, no inset
- have the same sub-elements: `grid.cell`, `grid.hline`, `grid.vline`

Questions

1. What is a sides dictionary? Give examples.
2. What is a joined value? Give examples.
3. What is binding? Give examples.
4. What is discarding? Give examples?
5. What is CSV? Give an example.
6. For each cell in the empty table give its coordinates:



7. How to access all the cells with `y` or `x` coordinate equal to a specific number thru `#show` rule?
8. Explain using single value as `#table` argument.
9. Explain using array as `#table` argument.
10. Explain using sides dictionary as `#table` argument.
11. Explain using binary closure as `#table` argument.
12. Can we mix the above forms of values? If yes, give examples, if no, explain why.

13. What will `#if <condition> {<expression>}` return if the `condition` is false? Where is this mechanism useful? Where does it cause problems?
14. What is default size of table column?
15. How does `fill:` argument work with an array? How does it work with `none`? How does it work with sides dictionary?
16. How does `stroke:` argument work with an array? How does it work with `none`? How does it work with sides dictionary?
17. How does `align:` argument work with an array? How does it work with `none`? How does it work with sides dictionary?
18. What kind of joined value can `stroke:` argument receive?
19. What does `inset:` argument do? Which values can it take in terms of type and form?
20. Explain `gutter:` argument. Which values can it take in terms of type and form? How does it work with an array?
21. The same question for `row-gutter` and `column-gutter` arguments.
22. Explain `table.header` and `table.footer` elements.
23. Where do `table.header` and `table.footer` repeat themselves? How to switch off the repetition.
24. How to access `table.header` or `table.footer` thru `#show` rule.
25. Explain `table.cell` element. Which arguments does it take? How to adjust it thru `#set` and `#show`?
26. How to make a cell span over several rows/columns?
27. Explain `table.hline` and `table.vline`.
28. Can we change `table.hline` and `table.vline` via `#set` and `#show` rules?
29. Explain difference between hand-placed lines and grid-lines.
30. Explain `stroke`, `x` and `y`, `start` and `end` arguments of table lines.
31. Which logical operators does Typst use for $\vee$, $\wedge$ and $\neg$?
32. How to adjust a variable using the variable recursively?
33. Which delimiters are used to evaluate an expression (e.g., 2*2) in markup?
34. How to scope definitions and `#set` rules?

35. Fix a bug:

```
#(1, 2, 3).map(x =>
  let double = x*2
  double + 1)
```

36. How to convert strings like `"333", "4", "61"` into numbers?
37. Explain `#calc.odd`, `calc.even`, `calc.rem`, `#calc.round`, `#calc.log`.
38. Where do array indexes begin from?
39. Explain `reflow:` argument.
40. Explain `<array>.at()` method?
41. How to make an array (for example an array of colors in `fill:` argument) cycle by rows instead of columns?
42. Explain `.flatten()` method.
43. Explain `.slice()` method.
44. Explain `#csv` function.
45. Explain destructuring with _ and with `..`
46. What is the output of `#(1, 2, 3).at(1)`
47. Explain `#rotate` function?
48. Explain `placement:` argument of `#figure`.
49. Why cannot a table span across pages when inside a figure? How to fix it?
50. Explain `#grid` function. What's its fundamental difference from `#table`? Which arguments it takes, what are defaults of these arguments?
51. What're grid's analogs of `table.cell, table.hline` and `table.vline`?